

Module 07: Interfaces

A class may extend exactly one class and implement any number of interfaces. Everything in this module follows from that asymmetry.

The module ends with the strongest argument against inheritance you will see in this curriculum, and it is not a slogan. It is two classes from the standard library giving different answers to identical code.

What You'll Learn

- Interfaces as capabilities, and why to accept the narrowest one that works
- When an interface beats an abstract class, and when it does not
- Default methods, and the specific problem they were invented to solve
- Static and private interface methods
- Why the diamond problem is a compile error rather than a guess
- What "prefer composition" actually protects you from

Coming From Another Language

	Python	Java
Contract	duck typing, or <code>Protocol</code>	<code>interface</code> , checked at compile time
Multiple parents	full multiple inheritance	one class, many interfaces
Mixins	via multiple inheritance	via interfaces with default methods
Conflict resolution	MRO decides silently	compile error, you decide

Python resolves ambiguity for you through the method resolution order. Java refuses to, on the grounds that a silent choice between two equally plausible behaviours is how you get bugs nobody can explain.

The Lesson

Interfaces are capabilities

From `Interfaces.java` :

```
private static void launch(Flies thing) {  
    System.out.println("launching: " + thing.fly());  
}
```

That method works on a duck, an aeroplane, and anything anyone writes later. It does not care what the object *is*, only that it can fly.

```
launching: duck flapping
launching: jet engines
```

A `Penguin` that implements `Swims` and `Speaks` but not `Flies` will not compile if passed in. The capability is checked, not assumed.

Accept the narrowest capability that does the job. A parameter typed `Flies` can take types that do not exist yet. One typed `Duck` cannot.

Interface or abstract class?

	interface	abstract class
How many per class	any number	exactly one
Instance fields	no	yes
Constructors	no	yes
State	no	yes

Ask what you are modelling. **"Can do" is an interface:** `Flies`, `Comparable`, `AutoCloseable`. **"Is a" is an abstract class**, and only when subclasses genuinely share state and construction logic.

When both fit, prefer the interface. It leaves the implementer's single inheritance slot free, and that slot is spent easily and regretted later.

Default methods, and why they exist

From `DefaultMethods.java`:

```
interface Collection {
    List<String> tracks();                // must implement

    default int count() { return tracks().size(); } // free
    default boolean isEmpty() { return count() == 0; }
}
```

The history matters here. Before Java 8, adding a method to an interface broke every existing implementation everywhere, immediately. Interfaces in public libraries were effectively frozen forever.

Java 8 needed to add `stream()` to `java.util.Collection`. Without default methods that one addition would have broken every collection class ever written by anyone.

Default methods exist for library evolution. They are not a way to sneak multiple inheritance in, and using them as one produces the next section.

Two smaller additions:

- **Private interface methods** (Java 9) let several defaults share a helper without exposing it to implementers.
- **Static interface methods** belong to the interface itself. This is why the library has `List.of`, `Map.of` and `Comparator.comparing`.

The diamond problem

Two interfaces, the same default, one class implementing both. `DiamondConflict.java`:

```
error: types Timestamped and Versioned are incompatible;
class Entry implements Timestamped, Versioned {
^
  class Entry inherits unrelated defaults for describe() from types Timestamped and
  Versioned
```

Java banned multiple class inheritance for twenty years to avoid exactly this. Default methods reintroduced a narrow version, so the language handles it the only safe way: it makes you decide.

```
@Override
public String describe() {
    return Timestamped.super.describe() + " / " + Versioned.super.describe();
}
```

The `InterfaceName.super.method()` syntax exists solely for this. There is no conflict when only one interface supplies a default, or when the class supplies its own.

What composition actually protects you from

This is the part worth remembering. `CompositionOverInheritance.java` writes one counting class three ways. Real output:

```
--- inheritance, extending HashSet ---
added 3, counter says: 6    <- wrong

--- the SAME code, extending ArrayList ---
added 3, counter says: 3    <- right, by luck

--- composition ---
added 3, counter says: 3    <- right, by design
```

The override is identical in both subclasses:

```
@Override public boolean add(String item)    { added++; return super.add(item); }
@Override public boolean addAll(Collection<? extends String> items) {
    added += items.size();
    return super.addAll(items);
}
```

`HashSet.addAll` calls `this.add` for each element, so the override runs again and the count doubles.

`ArrayList.addAll` copies in bulk, so it does not.

Two classes from the same library, the same code, different answers. And the subclass cannot see why from its own source.

That is the **fragile base class** problem. The correctness of your class depends on an implementation detail the parent never promised and may change in any release. Composition removes the dependency entirely: hold the collection, do not become one.

The cost is that you write the methods you want to expose rather than inheriting forty. That cost is usually worth paying, and it forces a good question: which of those forty did this type actually want to offer?

Run It

```
java modules/07-interfaces/examples/Interfaces.java
java modules/07-interfaces/examples/DefaultMethods.java
java modules/07-interfaces/examples/CompositionOverInheritance.java

# Fails on purpose.
java modules/07-interfaces/examples/DiamondConflict.java

./scripts/verify-examples.sh modules/07-interfaces
```

Common Mistakes

Typing a parameter to a class when an interface would do. It shuts out every type written after yours.

Using default methods to build mixins. They were made for library evolution. Interfaces still cannot hold state, so a "mixin" with anything to remember does not work anyway.

Extending a library class to add behaviour. The counting example is what happens. Wrap it instead.

Forgetting that interface methods are implicitly public. Implementing one with weaker visibility is a compile error, and the message is not obvious the first time.

Reaching for an abstract class because "there might be shared code later". Start with the interface. Adding an abstract class beneath it later is easy; reclaiming an implementer's inheritance slot is not.

Key Takeaways

- **One superclass, many interfaces.** That limit is the reason interfaces exist.
- **Accept the narrowest capability that does the job**, so future types can satisfy it.
- **"Can do" is an interface, "is a" is a class.** When both fit, take the interface.
- **Default methods exist so libraries can grow** without breaking implementers.
- **Conflicting defaults are a compile error**, resolved with `Interface.super.method()`.
- **Inheritance couples you to the parent's implementation**, not just its API. The same override is correct under `ArrayList` and wrong under `HashSet`.

Homework

[homework/README.md](#)

Design with capabilities, then reproduce the fragile base class bug and fix it by composing. Reference solution in [solutions/07-interfaces/](#).